資料分析與機器學習 (Data Analysis and Machine Learning)

透過特徵提取達到降低資料維度並保留最重要的資訊，進而提升模型的預測準確度與運算效率

Lecture 9 – Dimensionality Reduction and Applications

教師：陳慶永

Outline

- Dimensionality Reduction via Feature Extraction
- Linear Algebra: Matrix Eigenvalue Problem
  - Eigenvalues and eigenvectors of a symmetric matrix
- Linear Dimensionality Reduction
  - Unsupervised dimensionality reduction
    - Principal component analysis (PCA)
  - Supervised dimensionality reduction
    - Linear discriminant analysis (LDA)
- Nonlinear Dimensionality Reduction
  - t-distributed stochastic neighbor embedding (t-SNE)
- Classwork
  - Assignment #5

Dimensionality Reduction via Feature Extraction

Issues of High-dimensionality

- In ML, dimension equals the number of features (or input variables) that describe each data sample
- Curse of Dimensionality (維度災難) caused by High-dimensionality
  - Data sparsity (在高維空間中資料點會變得極其稀疏，模型很難找到資料之間的統計規律)
  - Loss of distance meaning (任兩點資料之間的距離都會變得非常相似)
  - Severe overfitting (特徵數量遠大於樣本數量時，模型會變得太過複雜)
  - Computational bottlenecks (訓練模型所需的記憶體與計算時間呈指數級增長)
  - Multi-collinearity among features (特徵之間存在高度相關性，或者純粹是雜訊)

Dimensionality Reduction

- We learned about reducing the dimensionality of a dataset via feature selection (特徵選取) techniques
  - *Select* a subset of the original features
  - Example:
    - Sequential Feature Selection
- An alternative approach to feature selection for dimensionality reduction is feature extraction (特徵提取)
  - *Derive* information from the feature set

Feature Extraction

- Can be understood as
  - An approach to data compression with the goal of maintaining *most* of the *relevant* information
  - *Summarize* the information content of a dataset by transforming (or projecting) it onto a *new* feature subspace (of *lower* dimensionality than the original one)
    - Improve storage space
    - Improve the computational efficiency of the learning algorithm
    - Improve the predictive performance

Linear Algebra: Matrix Eigenvalue Problem

Matrix Eigenvalue Problem

- Consider an $n \times n$ square matrix $\mathbf{A}$
- What is a Matrix Multiplication (i.e., linear transformation) doing?
  - Multiplication of a matrix by a vector
    - For most vectors $\mathbf{x}$, $\mathbf{Ax}$ changes both direction and magnitude
    - For *some* $\mathbf{v}$, $\mathbf{Av} = \lambda\mathbf{v}$
      - Direction remains invariant; only the scale changes
      - Those special $\mathbf{v}$ (vector) and corresponding λ (scalar value) characterize the matrix ($\mathbf{A}$ 的本命)

Matrix Eigenvalue Problem

- A special linear transformation in the form:

$$\mathbf{Ax} = \lambda\mathbf{x}$$ (stretching, compressing and flipping)

  - $\mathbf{Ax}$ should be proportional to $\mathbf{x}$
    - It produces a new vector with the same or opposite direction as the original vector
    - Scaling by a factor λ
  - λ: Eigenvalue (or characteristic value) (特徵值) of $\mathbf{A}$
  - $\mathbf{x} \neq \mathbf{0}$: Eigenvectors (or characteristic vectors) (特徵向量) of $\mathbf{A}$ corresponding to that eigenvalue λ
    - An eigenvector can be normalized (i.e., multiply it by a scalar) to get a normalized eigenvector (or unit eigenvector) of length 1 (unit vector)
    - Example: (表示方向; 大小為 1)

$\mathbf{x}_1 = [1 \quad 2]^T$ can be normalized to $[1/\sqrt{5} \quad 2/\sqrt{5}]^T$

The set of *all* the eigenvalues of $\mathbf{A}$ is called the spectrum of $\mathbf{A}$

Eigenbasis (特徵基底) (必須有 n 個獨立的特徵向量才能組成特徵基底)

- *If* an $n \times n$ square matrix $\mathbf{A}$ has n linearly independent eigenvectors $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, then these vectors constitute a basis, called eigenbasis (basis of eigenvectors), for $\mathbb{R}^n$
  - *Any* vector $\mathbf{x}$ in $\mathbb{R}^n$ can be represented as

$$\mathbf{x} = c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \cdots + c_n\mathbf{x}_n$$

(Linear combination of eigenvectors)

    - Linear transformation is then decomposed to a sum of simple actions

$$\begin{aligned}\mathbf{y} = \mathbf{A}\mathbf{x} &= \mathbf{A}(c_1\mathbf{x}_1 + \cdots + c_n\mathbf{x}_n) \\ &= c_1\mathbf{A}\mathbf{x}_1 + \cdots + c_n\mathbf{A}\mathbf{x}_n \\ &= c_1\lambda_1\mathbf{x}_1 + \cdots + c_n\lambda_n\mathbf{x}_n.\end{aligned}$$

複雜的線性轉換被分解 (簡化) 為數個獨立方向上的單純縮放 😮

研究特徵值與特徵向量，目標就是為了找到一個最完美的觀測角度 (特徵向量坐標系)，使得矩陣的行為變得清晰 (反應在特徵值)

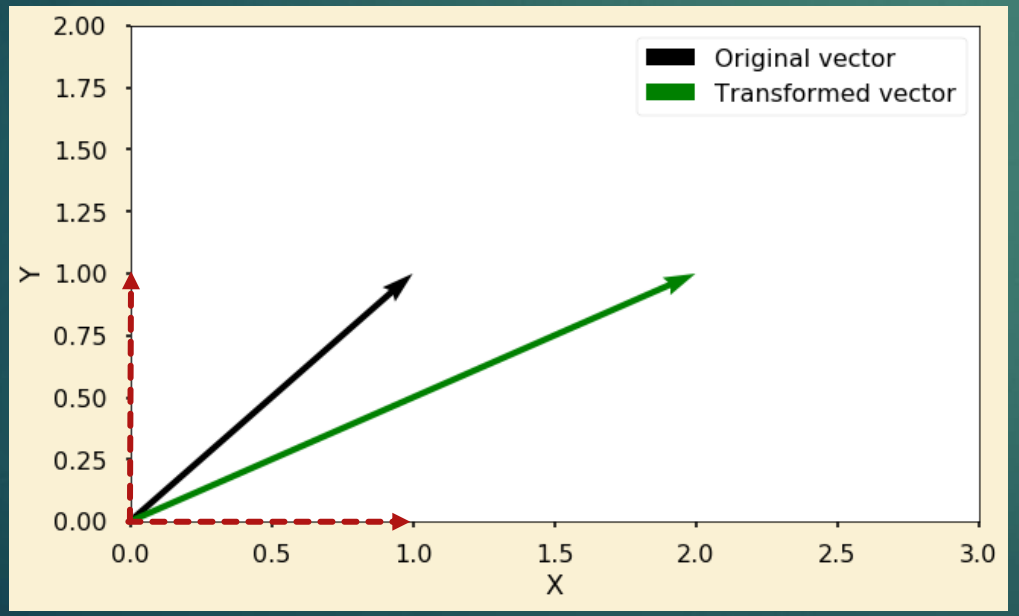
Meaning of Linear Transformation

$\mathbf{x}_1 = \begin{bmatrix}1\\0\end{bmatrix}, \mathbf{x}_2 = \begin{bmatrix}0\\1\end{bmatrix}$

$\lambda_1 = 2, \quad \lambda_2 = 1$

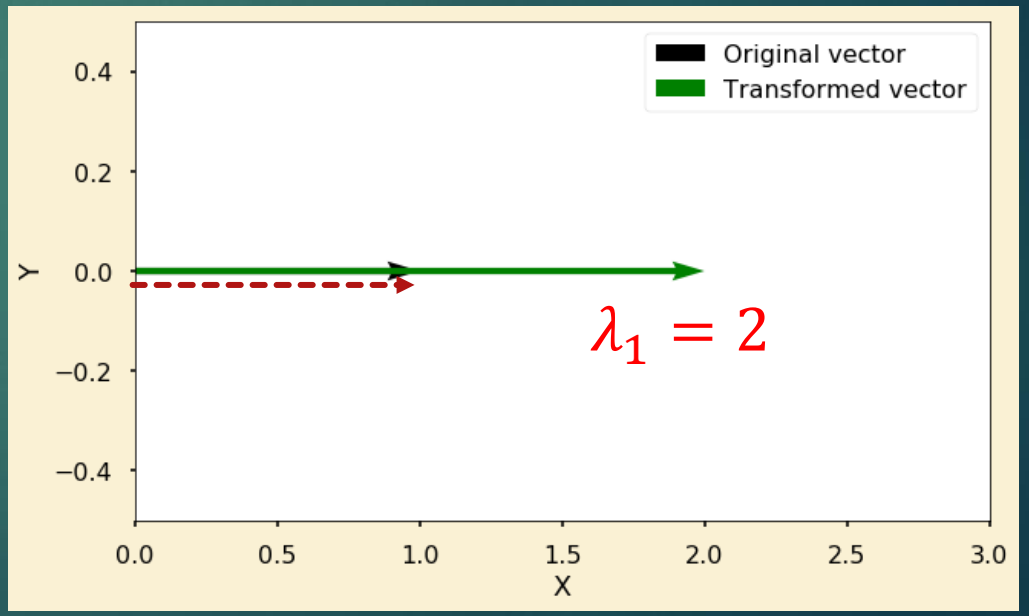
- Example: A = np.array([[2, 0],[0, 1]]) $\mathbf{A} = \begin{bmatrix}2 & 0\\0 & 1\end{bmatrix}$

```
x = np.array([[1],[1]])
b = np.dot(A, x)
plot_vect(x,b,(0,3),(0,2))
```




```
x = np.array([[1], [0]])
b = np.dot(A, x)

plot_vect(x,b,(0,3),(-0.5,0.5))
```



Symmetric Matrix (對稱矩陣)

- A real square matrix $\mathbf{A}$ satisfying

$$\mathbf{A}^T = \mathbf{A}$$ (轉置矩陣等於自己)

  - Example:

$$\begin{bmatrix} -3 & 1 & 5 \\ 1 & 0 & -2 \\ 5 & -2 & 4 \end{bmatrix}$$

  - Properties of a symmetric matrix
    - The eigenvalues are real
    - Has an orthonormal eigenbasis (標準正交特徵基底) for $\mathbb{R}^n$
      - Eigenvectors $\mathbf{x}_i \perp \mathbf{x}_j$ for $i \neq j$ and $\|\mathbf{x}_i\| = 1$

(正交向量保證線性獨立)

(對稱矩陣保證有特徵基底，而且一定可以轉化成一組標準正交基底)

Symmetric Matrix (對稱矩陣)

- Decomposition of a vector $\mathbf{x}$ in $\mathbb{R}^n$ using an orthonormal eigenbasis

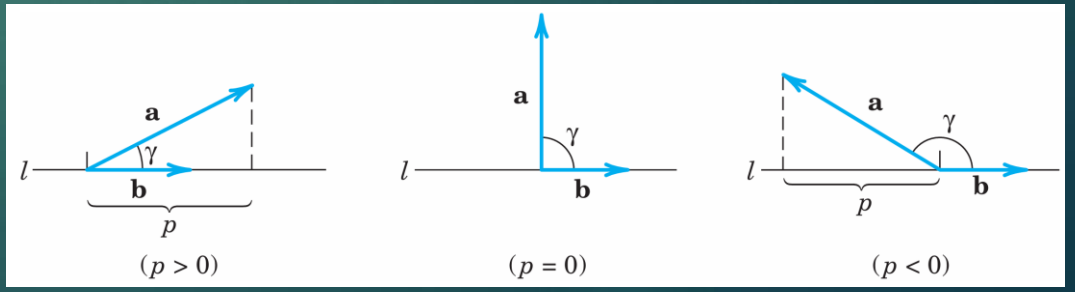
$$\mathbf{x} = c_1\mathbf{x}_1 + c_2\mathbf{x}_2 + \cdots + c_n\mathbf{x}_n$$

  - c_i is virtually the projection (投影) of $\mathbf{x}$ in the direction of the eigenvector $\mathbf{x}_i$ ($\because \mathbf{x}_i \perp \mathbf{x}_j$ for $i \neq j$ and $\|\mathbf{x}_i\| = 1$)

$$c_i = \mathbf{x} \cdot \mathbf{x}_i = \mathbf{x}^T\mathbf{x}_i$$

向量內積的幾何意義 (投影)

$$p = |\mathbf{a}| \cos\gamma = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{b}|}$$



$\frac{\mathbf{b}}{|\mathbf{b}|}$ is just a unit vector with the same direction as $\mathbf{b}$

Symmetric Matrix (對稱矩陣)

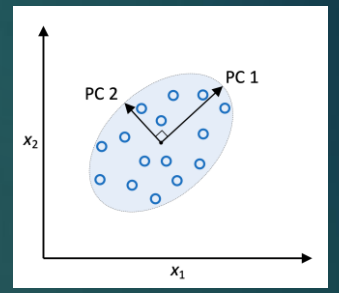
- Vector $\mathbf{x}$ is completely represented by a n-dimensional vector $[c_1, c_2, \ldots, c_n]$ in a space spanned by the orthonormal eigenbasis $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ (i.e., n eigenvectors of $\mathbf{A}$)
- Compression of the vector $\mathbf{x}$ using a *subset* of orthonormal eigenvectors of $\mathbf{A}$
  - Ex: $\tilde{\mathbf{x}} = c_i\mathbf{x}_i + c_j\mathbf{x}_j$ (using only 2 eigenvectors)
    - $\mathbf{x}$ is now compressed to a 2-dimensional vector $[c_i, c_j]$ in a subspace spanned by $\{\mathbf{x}_i, \mathbf{x}_j\}$ (i.e., two eigenvectors of $\mathbf{A}$)
    - Questions:
      - How to choose the subset of eigenvectors for maintaining the information in $\mathbf{x}$ *most relevant to* $\mathbf{A}$?
      - What is $\mathbf{A}$ in the sense of *feature extraction*?

Linear Dimensionality Reduction

(Unsupervised)

Principal Component Analysis (PCA)

- PCA (主成分分析) identify patterns in data based on the correlation (相關性) between features
  - Find the (mutually orthogonal) directions of maximum variance (i.e., most informative) in high-dimensional data
    - Those directions are called *principal components*
    - Highly sensitive to data scaling
      - We need to standardize the features prior to PCA



  - Projects the data onto a new subspace with fewer dimensions than the original one

Principal Component Analysis (PCA)

- PCA Algorithm

Number of samples

Number of features

  - Standardize the n-dimensional dataset, $\mathbf{X}$ $(N \times n)$
  - Construct the covariance matrix $\mathbf{R}_{xx}$ of $\mathbf{X}$
  - Decompose the covariance matrix into its eigenvectors $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ and eigenvalues $\{\lambda_1, \lambda_2, \ldots, \lambda_n\}$
  - Sort the eigenvalues by *decreasing order* (e.g., $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$) to rank the corresponding eigenvectors
  - Select k eigenvectors (e.g., $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_k\}$) which *correspond to the k largest eigenvalues*
    - k $(\leq n)$ is the dimensionality of the new feature subspace

See p. 19 for a brief review of the covariance matrix

Principal Component Analysis (PCA)

- PCA Algorithm
  - Construct a projection matrix, $\mathbf{W}$, from the "top" k eigenvectors

$$\underset{(n \times k)}{\mathbf{W}} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_k]$$

  - Transform the n-dimensional input dataset, $\mathbf{X}$, using the projection matrix, $\mathbf{W}$, to obtain the new k-dimensional feature subspace

$$\underset{(N \times k)}{\mathbf{X}'} = \underset{(N \times n)}{\mathbf{X}} \underset{(n \times k)}{\mathbf{W}}$$

(Linear transformation)

Covariance Matrix (共變異數矩陣)

- For a random vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$
  - The covariance matrix of $\mathbf{x}$ is defined as

$$E[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T] \quad \text{where } \boldsymbol{\mu} = E[\mathbf{x}]$$

  - It is a symmetric matrix where each entry (i, j) represents the covariance between x_i and x_j

$$\begin{bmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) & \ldots & \mathrm{Cov}(x_1, x_n) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Var}(x_2) & \ldots & \mathrm{Cov}(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(x_n, x_1) & \mathrm{Cov}(x_n, x_2) & \ldots & \mathrm{Var}(x_n) \end{bmatrix}$$

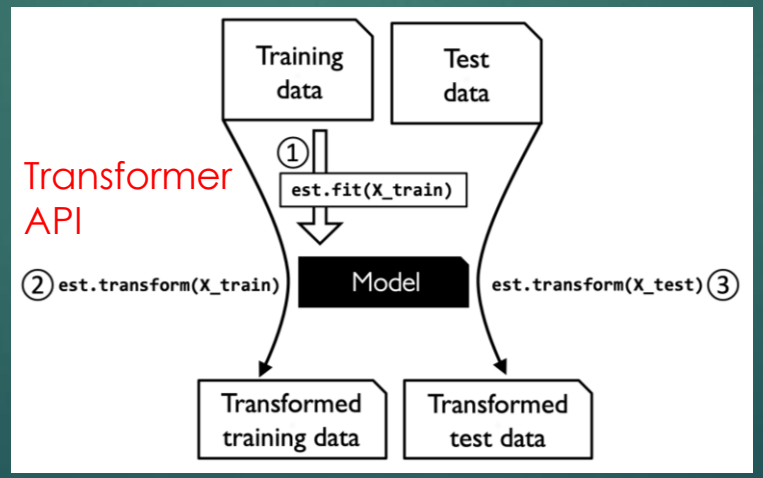
實務上會用總體平均值 (ensemble mean) 取代理論期望值 $E[\cdot]$

描述多個隨機變數之間「各自的離散程度」(variance)
以及「兩兩之間線性相關程度」(covariance)

將變數思考為特徵

PCA in scikit-learn

- sklearn.decomposition.PCA
  - One of scikit-learn's transformer classes
    - fit the model using the training data
    - transform both the training data and the test dataset using the same model parameters




```
pca = PCA(n_components=2)
pca.fit(X)
```

Dimensionality of the new feature subspace

Any data array to be transformed needs to have the same number of features as the data array that was used to fit the model.

PCA in scikit-learn

- Standardize the dataset

```
>>> # standardize the features
>>> from sklearn.preprocessing import StandardScaler
>>> sc = StandardScaler()
>>> X_train_std = sc.fit_transform(X_train)
>>> X_test_std = sc.transform(X_test)
```

- Dimensionality reduction using PCA

```
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.decomposition import PCA
>>> # initializing the PCA transformer and
>>> # logistic regression estimator:
>>> pca = PCA(n_components=2)
>>> lr = LogisticRegression(multi_class='ovr',
...                         random_state=1,
...                         solver='lbfgs')
>>> # dimensionality reduction:
>>> X_train_pca = pca.fit_transform(X_train_std)
>>> X_test_pca = pca.transform(X_test_std)
>>> # fitting the logistic regression model on the reduced dataset:
>>> lr.fit(X_train_pca, y_train)
```

PCA in scikit-learn

- Explained Variance Ratios (解釋變異量比率) of the eigenvalues (the magnitude of eigenvectors)
  - The fraction of an eigenvalue and the total sum of the eigenvalues

$$\text{Explained variance ratio} = \frac{\lambda_j}{\sum_{j=1}^{d} \lambda_j}$$

  - `pca.explained_variance_ratio_`
    - Example: `[0.43502782 0.19500007 0.09781519 0.06486409 0.05253378]`

Total explained variance (總解釋變異量) ≅ 84.52% for $k = 5$

PCA in scikit-learn

- Just apply PCA within a Pipeline
  - The role of PCA is feature extraction

```
>>> from sklearn.preprocessing import StandardScaler
>>> from sklearn.decomposition import PCA
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.pipeline import make_pipeline
>>> pipe_lr = make_pipeline(StandardScaler(),
...                         PCA(n_components=2),
...                         LogisticRegression())
>>> pipe_lr.fit(X_train, y_train)
>>> y_pred = pipe_lr.predict(X_test)
```

2 transformers

estimator

Applications of the PCA

- Mission
  - Dimensionality reduction using the PCA
    - Observe the total explained variance for various reduced dimensionality ($k = 2, 3, ...$)
  - Apply it to the Breast Cancer Wisconsin dataset (乳腺癌威斯康辛資料集)
- Sample code: 09_1.py

Applications of the PCA

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# 1. 載入 WDBC 資料集
wdbc = load_breast_cancer()
X = wdbc.data
y = wdbc.target
target_names = wdbc.target_names

# 2. 黃金法則：進行任何 Processing 之前，先切分資料集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. 資料標準化（PCA 對資料尺度非常敏感，降維前必須先標準化）
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

Applications of the PCA

```python
print(f"原本的資料形狀: {X_scaled.shape}\n")
print(f"{'目標維度 (k)':<12}{'總解釋變異量 (Total Explained Variance)':<30}")
print("-" * 50)

# 4. 迴圈測試 k=2 到 10，彙整總解釋變異量並輸出
for k in range(2, 11):
    # 初始化 PCA 並指定降維至 k 維
    pca = PCA(n_components=k)

    # 對標準化後的資料進行擬合
    pca.fit(X_scaled)

    # 計算總解釋變異量
    total_variance = pca.explained_variance_ratio_.sum()

    # 輸出結果
    print(f"{k:<20}{total_variance:.2%}")
```

Original dimension

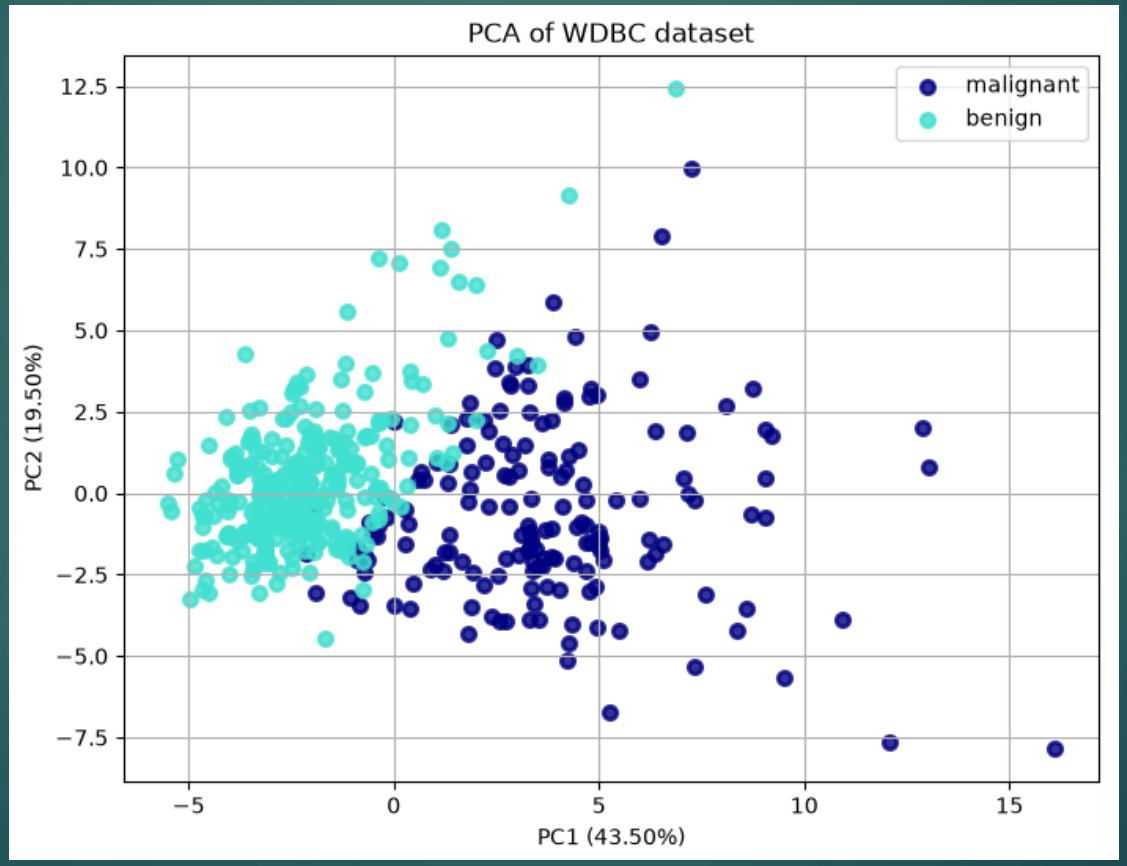
```
原本的資料形狀: (455, 30)

目標維度 (k)    總解釋變異量 (Total Explained Variance)
--------------------------------------------------
2                   63.00%
3                   72.78%
4                   79.27%
5                   84.52%
6                   88.64%
7                   90.87%
8                   92.52%
9                   93.90%
10                  95.11%
```

```
各主成分的解釋變異量比率: [0.43502782 0.19500007 0.09781519 0.06486409 0.05253378 0.041128
 0.0223559  0.01647952 0.01380052 0.01205262]
```

Applications of the PCA

- Visualization of the 2D feature space via PCA



Linear Dimensionality Reduction

(Supervised)

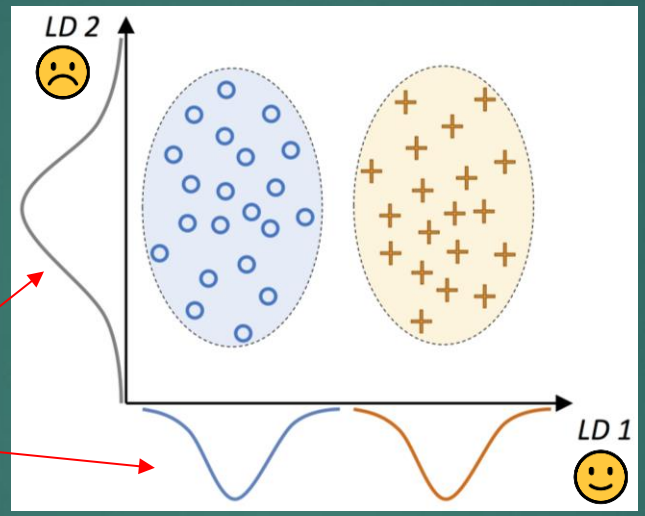
Linear Discriminant Analysis (LDA)

- The concept behind LDA (線性判別分析) is very *similar* to PCA
  - Both PCA and LDA are linear transformation techniques that can be used to reduce the number of dimensions in a dataset
- PCA is Unsupervised
  - Find the orthogonal component axes of maximum variance in a dataset (尋找特徵的線性組合，盡可能地保留最多的資訊)
- LDA is Supervised
  - Find the feature subspace that optimizes class separability

(尋找特徵的線性組合，盡可能地把不同類別分離)

Linear Discriminant Analysis (LDA)

- Concept of LDA for a two-class problem



(投影量的機率分布)

  - LD 1 would separate the two normal distributed classes well
  - Although the LD 2 captures a lot of the *variance* in the dataset, it would fail as a good linear discriminant since it does *not* capture any of the class-discriminatory information

Linear Discriminant Analysis (LDA)

- LDA Algorithm
  - Standardize the n-dimensional dataset, $\mathbf{X}$ $(N \times n)$
  - For *each class* i, compute the n-dimensional *mean vector*, $\boldsymbol{\mu}_i = \frac{1}{n_i}\sum_{\mathbf{x}\in D_i}\mathbf{x}$ (第 i 類的平均特徵)
    - Construct the *between-class* scatter matrix, $\mathbf{S}_B$, and the *within-class* scatter matrix, $\mathbf{S}_W$
    - Compute the eigenvectors and corresponding eigenvalues of the matrix, $\mathbf{S}_W^{-1}\mathbf{S}_B$

    與PCA的差異處
  - Sort the eigenvalues by *decreasing order* (e.g., $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$) to rank the corresponding eigenvectors
  - Select $k \leq \min(C-1, n)$ eigenvectors (e.g., $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_k\}$) which *correspond to the* k *largest eigenvalues*

Linear Discriminant Analysis (LDA)

- LDA Algorithm
  - Construct a projection matrix, $\mathbf{W}$, from the "top" k eigenvectors

$$\underset{(n \times k)}{\mathbf{W}} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_k]$$

  - Transform the n-dimensional input dataset, $\mathbf{X}$, using the projection matrix, $\mathbf{W}$, to obtain the new k-dimensional feature subspace

$$\underset{(N \times k)}{\mathbf{X}'} = \underset{(N \times n)}{\mathbf{X}} \underset{(n \times k)}{\mathbf{W}}$$

(Linear transformation)

Scatter Matrix (散布矩陣)

- Within-class (類別內，即自身) scatter matrix

單一類別內部點與點之間的密集程度

$$\mathbf{S}_W = \sum_{i=1}^{C} \mathbf{S}_i, \qquad \mathbf{S}_i = \frac{1}{n_i} \sum_{\mathbf{x} \in D_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T$$

- Between-class (類別間，即交互) scatter matrix

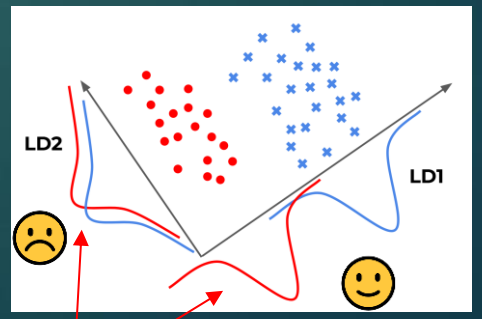
不同類別中心點之間的距離

$$\mathbf{S}_B = \sum_{i=1}^{C} n_i \underbrace{(\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^T}_{\text{rank} = 1}, \qquad \boldsymbol{\mu} = E[\mathbf{x}]$$

$\text{rank} \leq C$

實務上會用總體平均值 (ensemble mean) 取代理論期望值 $E[\cdot]$

- The meaning of largest eigen-pairs of $\mathbf{S}_W^{-1}\mathbf{S}_B$
  - *Large* distance *between* different class
  - *Small* distance *within* a single class



(投影量的機率分布)

LDA vs PCA

特性	LDA (線性判別分析)	PCA (主成分分析)
主要目標	**分類 (Classification)** 導向。	**資訊保留 (Information)** 導向。
物理視角	尋找能**完美切開不同類別**的投影方向。	尋找數據**變異數最大（最分散、細節最多）**的方向。
標籤需求	**監督式**（必須知道每個點是哪一類）。	**非監督式**（不需要知道標籤，只看整體形狀）。
缺點限制	如果數據本質上無法用直線切開（如非線性環狀分布），效果很差。	可能會為了保留最大變異，而把不同類別的特徵混在一起。

LDA in scikit-learn

- sklearn.discriminant_analysis.LinearDiscriminantAnalysis
  - One of scikit-learn's transformer classes
    - fit the model using the training data
    - transform both the training data and the test dataset using the same model parameters

```
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
>>> lda = LDA(n_components=2)
>>> X_train_lda = lda.fit_transform(X_train_std, y_train)
```

(Supervised)

$\leq \min(C-1, n)$

Applications of the LDA

- Mission
  - Dimensionality reduction using the LDA
  - Apply it to the Breast Cancer Wisconsin dataset (乳腺癌威斯康辛資料集)
    - Reduced dimensionality $k = 1$ (because $C = 2$)

- Sample code: 09_2.py

Applications of the LDA

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
import matplotlib.pyplot as plt
import numpy as np

# 1. 載入 WDBC 資料集
wdbc = load_breast_cancer()
X = wdbc.data
y = wdbc.target
target_names = wdbc.target_names

# 2. 黃金法則：進行任何 Processing 之前，先切分資料集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. 資料標準化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
```

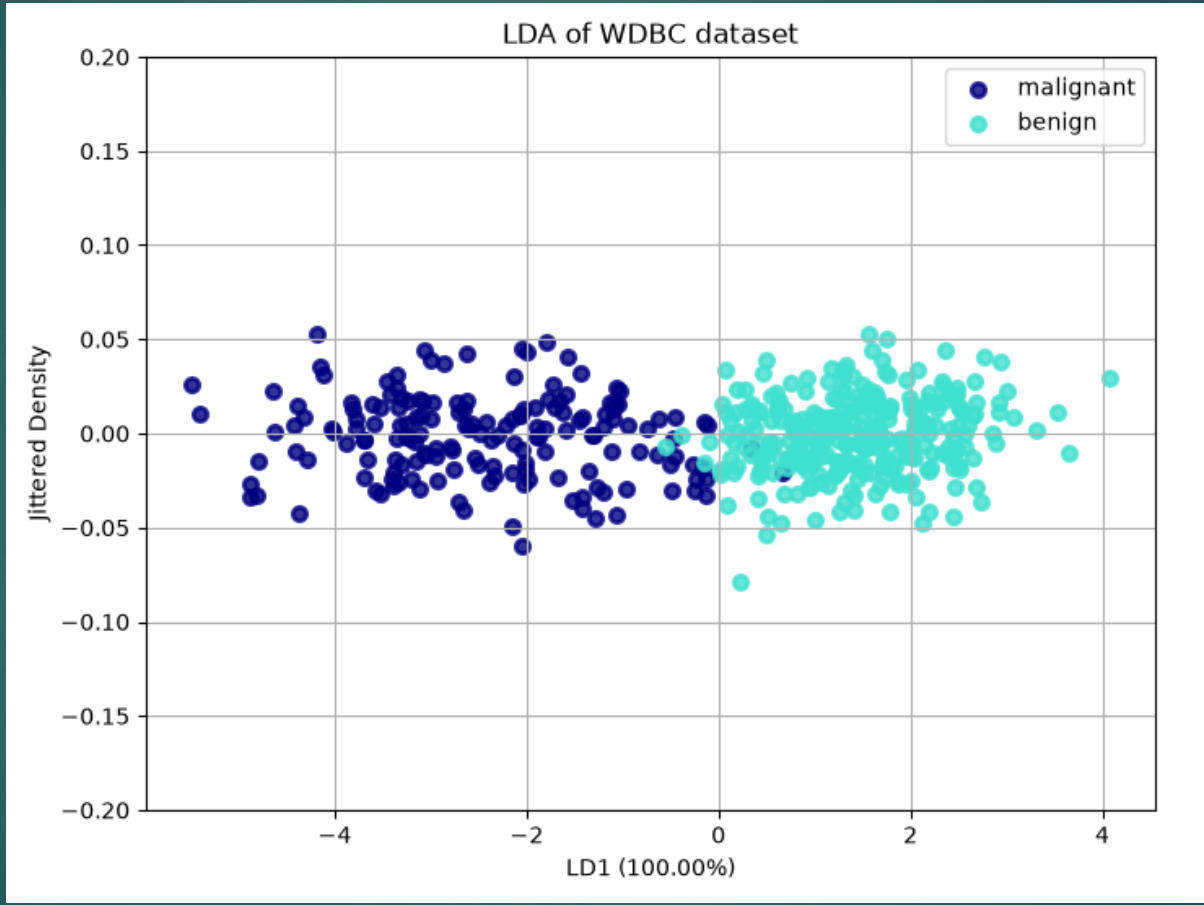
Applications of the LDA

```
23    # 4. 初始化 LDA
24    # 注意：因為 WDBC 只有 2 個類別 (M/B)，n_components 最大只能設定為 1 (類別數 - 1)
25    lda = LDA(n_components=1)
26
27    # 5. 對標準化後的資料進行擬合與轉換
28    # 監督式學習：fit_transform 必須同時傳入 X_scaled 與 y_train
29    X_lda = lda.fit_transform(X_scaled, y_train)
30
31    # 6. 輸出 LDA 重要的分析結果
32    print(f"原本的資料形狀: {X_scaled.shape}")
33    print(f"降維後的資料形狀: {X_lda.shape}\n")
34
35    # 解釋變異量比率 (Explained Variance Ratio)
36    # 在 LDA 中，這代表該判別軸區分類別的能力佔總區分能力的比例
37    print(f"各成分的解釋變異量比率: {lda.explained_variance_ratio_}")
38    print(f"總共保留的資訊量 (總解釋變異量): {lda.explained_variance_ratio_.sum():.2%}\n")
```

```
原本的資料形狀: (455, 30)
降維後的資料形狀: (455, 1)

各成分的解釋變異量比率: [1.]
總共保留的資訊量 (總解釋變異量): 100.00%
```

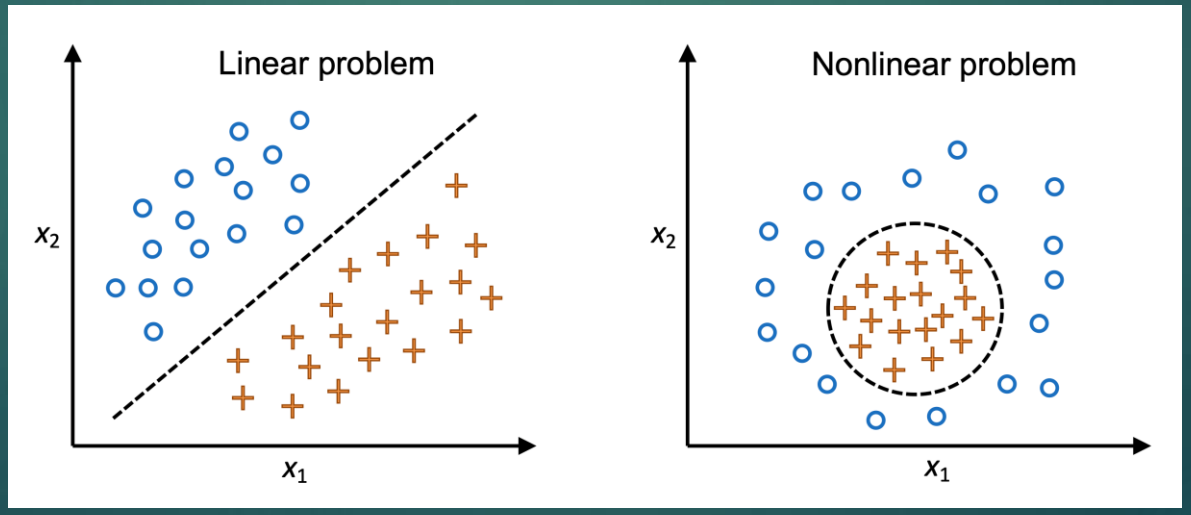
Applications of the LDA



Nonlinear Dimensionality Reduction

Why Nonlinear Dimensionality Reduction?

- If we are dealing with nonlinear problems
  - Rather frequently in real-world applications
  - Linear transformation techniques for dimensionality reduction (such as PCA and LDA) may not be the best choice



Manifold Learning (流形學習)

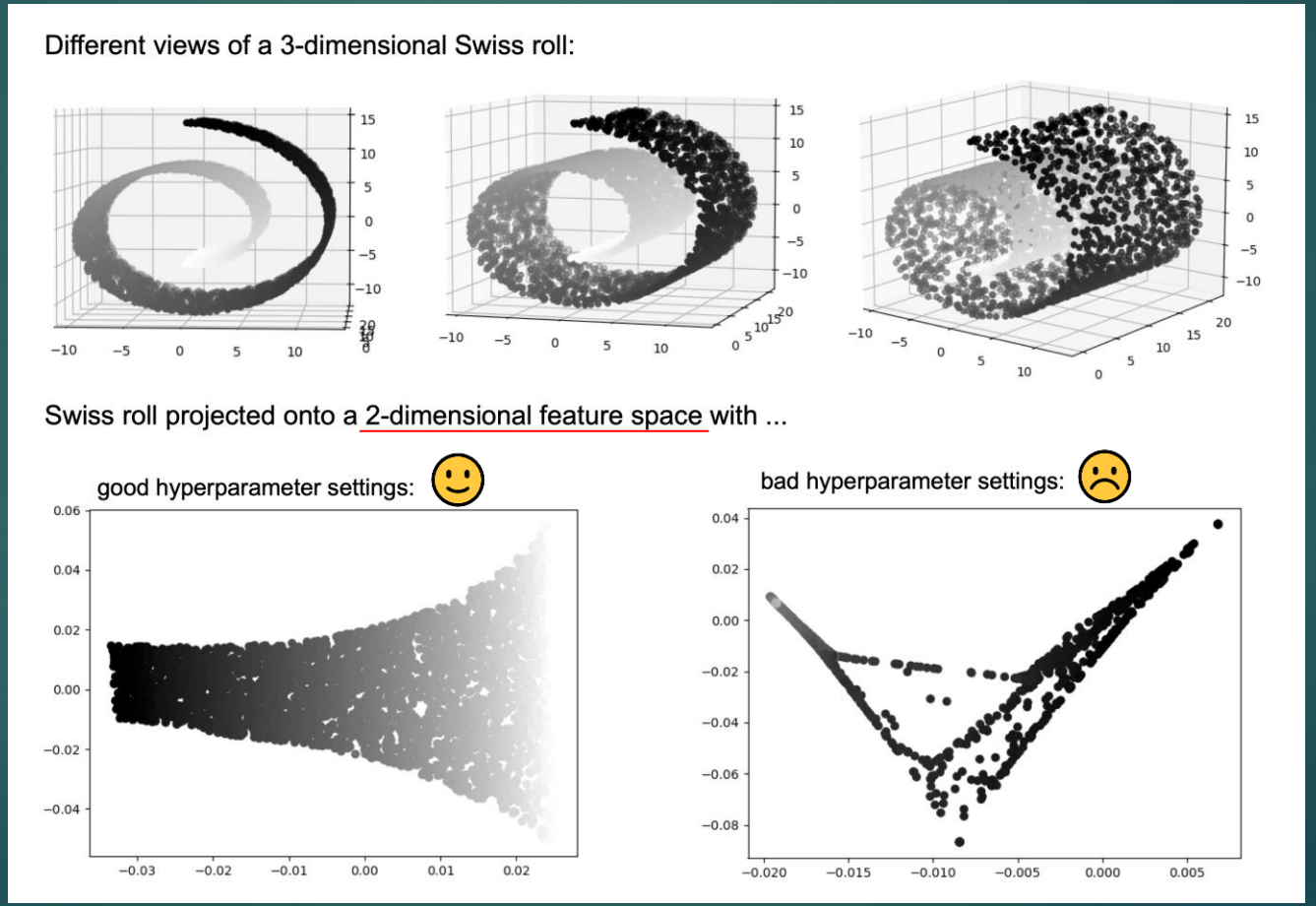
- The development and application of nonlinear dimensionality reduction techniques is also referred to as manifold learning
  - A *manifold* refers to a lower dimensional topological space embedded in a high-dimensional space
  - Algorithms for manifold learning have to capture the complicated *structure* of the data in order to project it onto a lower-dimensional space where the relationship between data points is preserved
    - Example: t-distributed stochastic neighbor embedding (t-SNE)
  - Used for visualization or feature extraction

將高維度資料對應到低維度空間的非線性降維方法，目的是在去除冗餘資訊的同時保留資料原本的幾何結構

An Example of Manifold Learning

- The 3-dimensional Swiss roll

非線性降維



Challenges of Manifold Learning

- We are often working with high-dimensional datasets (unlike the Swiss roll example)
  - Cannot readily visualize
  - The structure is not obvious
- Manifold learning techniques are hard to use
  - Non-ideal hyperparameter choices may cause more harm than good
  - Moreover, unless we project the dataset into two or three dimensions (which is often *not sufficient* for capturing more complicated relationships), it is hard or even impossible to assess the quality of the results
- Hence, many people still rely on simpler techniques such as PCA and LDA for dimensionality reduction

t-SNE

- Modeling data points based on their pair-wise distances in the high-dimensional (original) feature space
  - It finds a probability distribution of pair-wise distances in the new, lower-dimensional space that is *close to* the probability distribution of pair-wise distances in the original space
  - In other words, t-SNE learns to embed data points into a lower-dimensional space such that the pairwise distances in the original space are preserved
- t-SNE is a technique intended for visualization purposes as it requires the whole dataset for the projection
  - Since it projects the points directly (unlike PCA, it does not involve a projection matrix), we cannot apply t-SNE to new data points

將資料點在高維和低維空間的相似度轉換成機率分佈，應用於高維資料的視覺化

t-SNE in scikit-learn

- sklearn.manifold.TSNE
  - Only has the two method
    - fit
    - fit_transform

```
>>> from sklearn.manifold import TSNE
>>> tsne = TSNE(n_components=2, init='pca',
...             random_state=123)
>>> X_digits_tsne = tsne.fit_transform(X_digits)
```

Applications of the t-SNE

- Mission
  - Visualization of complex dataset using the t-SNE
  - Apply it to the 64-dimensional dataset, Digits (低解析度手寫數字資料集)
    - sklearn.datasets.load_digits()
    - Low-resolution handwrit ten digits (0-9)

- Sample code: 09_3.py

Applications of the t-SNE

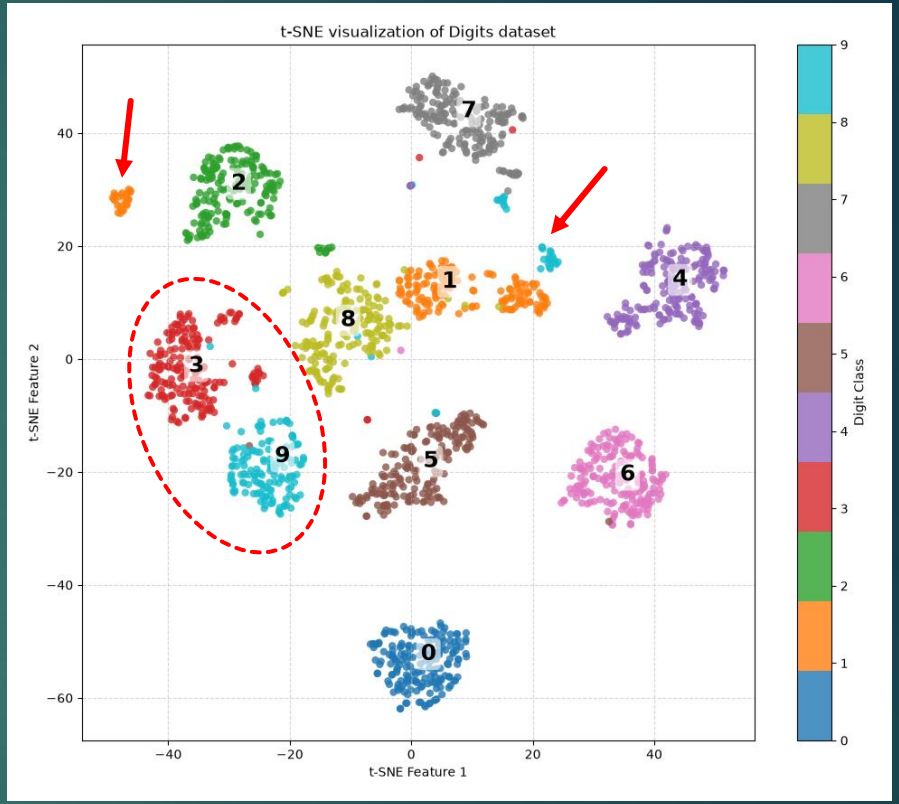
```python
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import numpy as np

# 1. 載入 Digits 手寫數字資料集 (64維，低解析度 8x8 影像)
digits = load_digits()
X = digits.data
y = digits.target
target_names = [str(i) for i in range(10)]

# 2. 初始化 t-SNE
# 將 64 維降至 2 維以利視覺化，設定 random_state 確保結果可重複
tsne = TSNE(n_components=2, init='pca', random_state=42)

# 3. 對資料進行擬合與轉換
X_tsne = tsne.fit_transform(X)

# 4. 輸出降維前後的結果
print(f"原本的資料形狀: {X.shape}")
print(f"降維後的資料形狀: {X_tsne.shape}\n")
```




```
原本的資料形狀: (1797, 64)
降維後的資料形狀: (1797, 2)
```

Classwork

Assignment #5

- 本次作業基本上與 Assignment #3 (Lecture 6) 的實作任務一樣，除了…
  - 將特徵選取換成特徵提取 (使用 PCA)

Assignment #5

- 目的：低解析度手寫數字 (0 - 9) 多類別分類器
- 資料集：sklearn.datasets.load_digits()
  - 1797 張 8 × 8 灰階 (grayscale) 影像
  - 64 維 (像素值) 特徵 (整數值 [0,16])
  - 多類別標籤 (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) (共 10 類)

Assignment #5

- 實作任務與要求：
  - 以 Logistic Regression 作為分類器演算法
  - 運用資料切分、特徵縮放、特徵提取 (使用 PCA)、模型選取與模型評估
    - 切分比例：80% 訓練 (用於交叉驗證)、20% 測試
    - 降維後的特徵空間維度 ≤ 40
    - 在測試集 (約 360 筆樣本) 上的整體正確率 (Accuracy) 達 95% 以上
  - 使用 joblib.dump() 將訓練好的最佳模型 (指管線) 儲存成檔案

```
import joblib
best_model = grid_search.best_estimator_
model_filename = "digits_pipeline_lr_pca.pkl"
joblib.dump(best_model, model_filename, compress=3)
print(f"成功將管線（含預處理與模型）儲存至: '{model_filename}'")
```

Assignment #5

- 上傳繳交作業要求：
  - 將模型評估部分的程式碼 (及模型儲存檔案) 獨立出來進行上傳繳交作業 (打包成單一 .zip 壓縮檔)
    - 不要包含任何訓練或模型選取程式碼
    - 助教將透過執行模型評估程式批閱作業 (沒有時間進行交叉驗證與微調)

```
1   import joblib
2   from sklearn.model_selection import train_test_split
3   from sklearn.metrics import classification_report, confusion_matrix
4   import matplotlib.pyplot as plt
5   plt.style.use('seaborn-v0_8-poster')
6
7   from sklearn.datasets import load_digits
8   # 1. 載入資料集
9   digits = load_digits()
10  X = digits.data
11  y = digits.target
```

僅載入 train_test_split() 與 metrics 相關函式

```
13  # 2. 切分訓練集與測試集 (80% 訓練, 20% 測試)
14  _, X_test, _, y_test = train_test_split(
15      X, y, test_size=0.2, random_state=42
16  )
17
18  # 3. 載入模型
19  model_filename = "digits_pipeline_lr_pca.pkl"
20  best_model = joblib.load(model_filename)
21  print("模型與預處理管線載入成功！")
22
23  # 4. Model Evaluation
24  y_pred = best_model.predict(X_test)
```

僅使用測試集資料

直接載入模型進行評估

Assignment #5

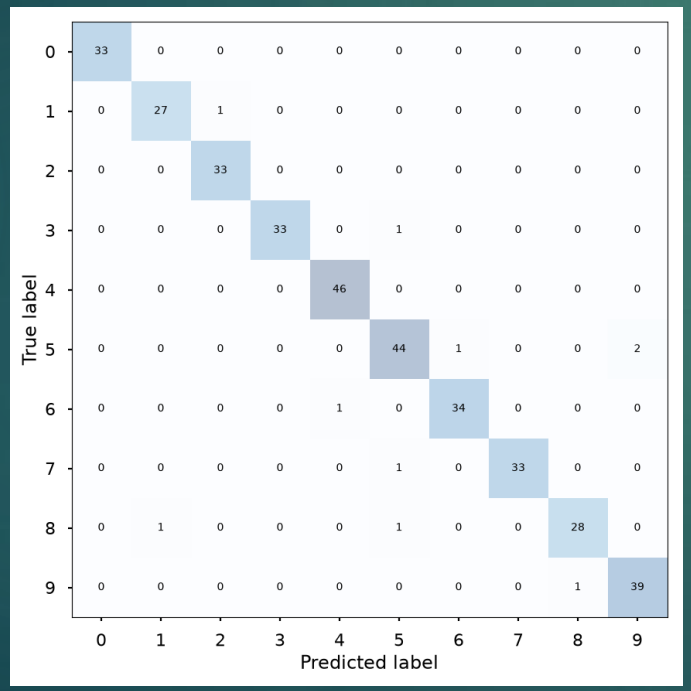
- 預期結果 1
  - 輸出 PCA 降維後的特徵空間維度
  - 輸出特徵提取的總解釋變異量

必須在 40 以下

```
PCA 提取後的特徵維度 (n_components): 40 (從原本的 64 維降至 40 維)
特徵提取的總解釋變異量: 95.18%
```

Assignment #5

- 預期結果 2
  - 輸出測試集的分類報告
  - 視覺化呈現混淆矩陣



詳細分類報告 (Classification Report):

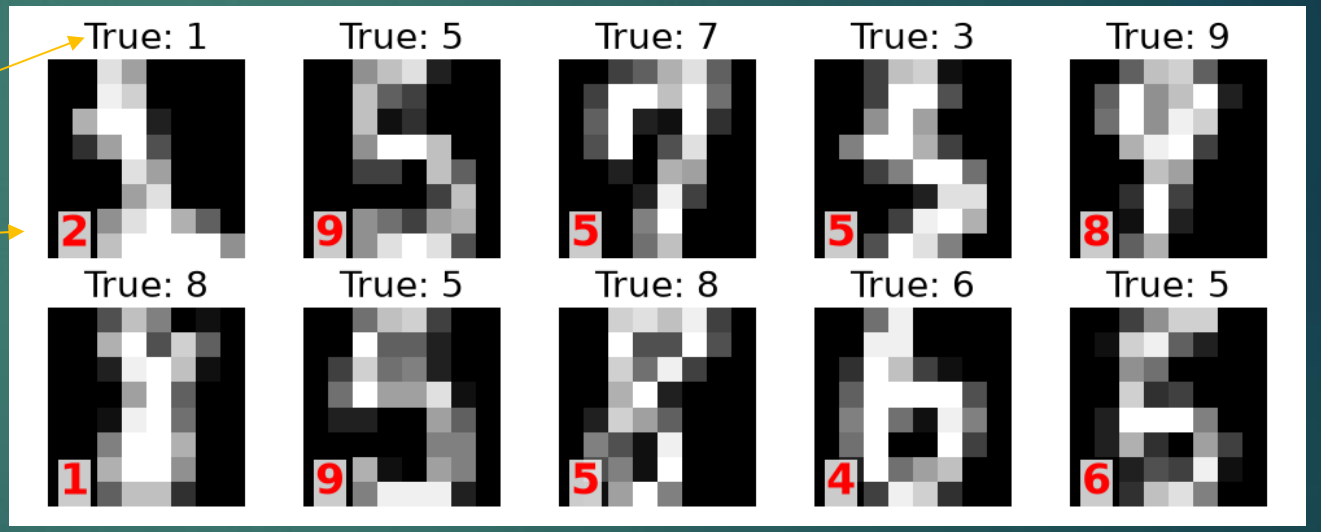
	precision	recall	f1-score	support
0	1.00	1.00	1.00	33
1	0.96	0.96	0.96	28
2	0.97	1.00	0.99	33
3	1.00	0.97	0.99	34
4	0.98	1.00	0.99	46
5	0.94	0.94	0.94	47
6	0.97	0.97	0.97	35
7	1.00	0.97	0.99	34
8	0.97	0.93	0.95	30
9	0.95	0.97	0.96	40
accuracy			0.97	360
macro avg	0.97	0.97	0.97	360
weighted avg	0.97	0.97	0.97	360

必須達 0.95 以上

Assignment #5

- 預期結果 3
  - 輸出誤判的樣本總數
  - 繪出所有誤判的樣本
    - 標示真實標籤
    - 標示預測結果
    - 必須與混淆矩陣對應

測試集總共誤判了 10 張圖片。



Thank you